AML Assignment 3

Name: Shruti Sharma (MDS202435)
Name: Riya Shyam Huddar (MDS202431)

Introduction

This project focuses on designing and training a reinforcement learning agent to solve the **LunarLander-v3 (Discrete)** environment from the Gymnasium library. The objective is to teach the agent to control a lander using thruster actions and perform a stable landing on a designated pad. The environment provides an 8-dimensional continuous state vector, and the agent must learn a policy that maps these states to both discrete (Case 1) and continuous (Case 2) actions.

Case 1 (Discrete actions):

1. Environment Details

- **Environment:** LunarLander-v3 (Discrete)
- **State (8 values):** x/y position, velocities, angle, angular velocity, and leg contacts
- **Actions (4):** do nothing, fire left, fire main, fire right engine
- **Reward Structure:** +100 to +140 for a successful landing; −100 for crashing; Small penalties for sideways motion, fuel usage, and unstable orientation
- **Termination:** The episode terminates when the lander crashes, lands safely, or exceeds time limits.

2. Deep Q-Network (DQN) Architecture

A neural network is used to approximate the action-value function Q(s,a):

- **Input dimension:** 8 (state vector)
- **Two fully connected layers** of 128 units each
- **ReLU** activation
- **Output dimension:** 4 (Q-value for each discrete action)
- **Weight initialization:** Kaiming (hidden layers), Xavier (output layer)

This architecture enables the network to learn nonlinear relationships between the state and optimal actions.

3. Reinforcement Learning Strategy

To stabilize Q-learning with function approximation, the following components were used:

- **Replay Memory** (capacity: 100,000): stores transitions (s,a,r,s′)

- **Target Network:** a periodically updated copy of the policy network (every 1000 steps) to prevent target drift.
- **Epsilon-Greedy Exploration:** ε starts at 1.0 (fully random), then decays exponentially to 0.05 over 40,000 steps, to encourage exploration early and exploitation later.
- **Prefill Phase:** The replay buffer is initially filled with 1000 random transitions before training begins.

4. Training Procedure

The agent was trained for **1000 episodes**. Each episode:

- Starts with a reset environment.
- Chooses actions using ε-greedy policy.
- Stores transitions in replay memory.
- Performs gradient updates using mini-batches of size 64.
- Syncs the target network periodically.

Training progress was monitored by printing average reward every 10 episodes, along with current ε and replay buffer size. Over time, the agent's total episode rewards increased, and the lander learned to hover, control descent velocity, and approach the landing pad more reliably.

5. Evaluation

After training, the agent evaluated over **20 deterministic episodes** using a greedy (non-exploratory) policy. During evaluation:

- No learning occurs
- No random actions are taken
- The agent relies purely on the learned Q-values

The agent achieved consistently positive rewards and multiple successful landings with **mean reward 210.87**, demonstrating that the DQN was able to learn stable control policies from scratch.

Case 2 (Continuous actions):

1. Environment Details

- **Environment:** LunarLander-v3 (Continuous)
- **State (8 values):** x/y position, velocities, angle, angular velocity, and leg contacts
- **Actions (2, continuous):** Main engine and side engines with real-valued thrust in [-1, 1]

2. **Deep Deterministic Policy Gradient (DDPG) Architecture (Off-policy learning)**

Two separate neural networks are used: the **Actor** for action selection and the **Critic** for value estimation.

Actor Network

- Input dimension: 8 (state vector)
- Two fully connected layers of 256 units each
- ReLU activation
- Output dimension: 2 (continuous actions for main and side engines)
- Tanh activation on output to bound actions in [-1, 1], scaled by max action
- Final layer weights initialized small for stability

Critic Network

- Input dimension: 10 (state + action concatenated)
- Two fully connected layers of 256 units each
- ReLU activation
- Output dimension: 1 (Q-value)

3. Reinforcement Learning Strategy & Training:

The DDPG agent uses a replay buffer (1,000,000 transitions) and Gaussian exploration noise (σ decays 0.2 to 0.05) for stable off-policy learning. Training begins after collecting 10,000 random transitions. The agent was trained up to 1000 episodes, but training terminated early once the **moving average return over the last 100 episodes exceeded 200**. In each episode, the agent resets the environment, selects actions via the actor network with noise, stores transitions in the replay buffer, performs mini-batch gradient updates on actor and critic (batch size 256), and softly updates the target networks after each step.

4. Key Results

- **Environment solved:** Episode 869, with an **average score of 202.62** over the last 100 episodes.
- **Policy evaluation (30 deterministic episodes): Average return = 261.82**, **Standard deviation = 22.51** implying mostly stable, high-performance landings.
- **Extensive evaluation (1000 deterministic episodes):** Average return = 245.80, Standard deviation = 64.87, Maximum = 323.56, Minimum = -42.96 , generally stable performance with occasional rare mislandings.
- The DDPG agent learned an effective continuous control policy, achieving smooth, consistent, and reliable landings.